

KINGA Claims Manager Portal — Alignment & Redesign Audit v3.0

Document Classification: Internal Engineering Audit

Portal Under Review: Claims Manager Portal (/insurer/claims-manager)

Audit Version: 3.0 — Operational Alignment and Redesign Review

Prepared by: KINGA Platform Engineering

Date: June 2026

Status: Final

Executive Summary

This audit assesses whether the KINGA Claims Manager Portal supports the real-world operational responsibilities of a Claims Manager managing a claims department within a short-term insurer. It does not revisit previously documented bugs. It does not compare the portal to the Executive Dashboard. Its singular purpose is to determine whether the portal functions as a **Claims Operations Command Centre** — a tool that enables a Claims Manager to manage claims throughput, queue health, workforce performance, approval governance, fraud oversight, and recovery oversight from a single operational workspace.

The assessment finds that the portal's foundational architecture is sound. The tab structure, workflow engine integration, and fraud alert visibility are well-designed. However, the portal currently operates as a **claims list viewer with approval capability** rather than a command centre. The Claims Manager can see claims and act on individual ones, but cannot see the operational health of the department at a glance. The analytics infrastructure required to support command-centre capability exists in the backend and is simply not surfaced.

Revised Overall Score: 5.8 / 10

The portal is operationally functional but not operationally intelligent. The gap between what the backend can provide and what the dashboard surfaces represents the primary engineering opportunity.

Phase 1 — Role Alignment Assessment

1.1 The Claims Manager's Operational Questions

A Claims Manager operating a claims department asks ten questions every working day. This phase assesses whether the current portal helps answer each one.

Operational Question	Supported?	Evidence
Which claims require my attention today?	Partial	Review Queue tab surfaces claims awaiting action, but no priority ranking or age-based sorting
Which queues are overloaded?	No	No queue-level count, age, or SLA status is visible anywhere on the dashboard
Which claims are breaching SLA?	No	No SLA breach indicator exists on any tab or KPI card
Which workflow stages are slowing down?	No	<code>workflowAnalytics.getBottlenecks</code> and <code>getProcessingTimesByStage</code> exist but are not called
Which claims require escalation?	Partial	Fraud Alerts tab surfaces high-risk claims; no stuck-claim or high-value escalation centre exists
Which assessors are overloaded?	No	<code>analytics.getAssessorPerformance</code> exists but is not called from this dashboard
Which processors are overloaded?	No	<code>analytics.getUserProductivity</code> exists but is not called from this dashboard
Which claims are waiting for approval?	Yes	Review Queue tab shows claims at <code>technical_approval</code> and <code>financial_decision</code> states
Which fraud claims require intervention?	Yes	Fraud Alerts tab surfaces high-risk and critical claims with review and escalate actions
Which recoveries require intervention?	Partial	Recovery KPI row shows open cases and demand sent counts, but no intervention trigger exists

Role Alignment Score: 4 of 10 questions fully supported. Six of the ten core operational questions receive no answer from the current dashboard. The four that are answered relate to individual claim review and fraud visibility — the reactive functions. The proactive management functions (queue health, SLA, bottlenecks, workforce) are entirely absent.

1.2 Widget Alignment Audit

Every current dashboard component is assessed against the ten operational questions above.

Widget	Answers Which Question	Verdict
KPI Cards (Total, Active, Completed, Fraud Alerts)	Partial: claims requiring attention	Informational — no drill-down to actionable queue
Recovery KPI Row (Open, Demand Sent, Settled)	Partial: recoveries requiring intervention	Informational — no link to recovery case list
Intake Queue Tab (IntakeQueueTab component)	Processor assignment	Actionable — assign to processor
Review Queue Tab (Claims Review Queue card)	Claims waiting for approval	Actionable — close for processing, send back
Active Claims Tab	Which claims are active	Partially actionable — view and route only
Fraud Alerts Tab	Which fraud claims require intervention	Actionable — review and escalate
Processed Claims Tab	Historical throughput	Informational only
Fleet Approvals Tab	Fleet-specific approvals	Actionable — approve fleet claims
Recently Closed Claims card	Historical closure confirmation	Informational only

Finding: Three of nine dashboard sections are fully actionable. Four are informational only. Two are partially actionable. No section addresses queue health, SLA compliance, workflow bottlenecks, or workforce performance.

Phase 2 — Command Centre Assessment

2.1 Pillar 1 — Queue Health

A Claims Operations Command Centre must show queue health immediately upon login. The following table defines the required queue health matrix and assesses current availability.

Queue Stage	Count Available	Avg Age Available	SLA Breached Available	UI Implementation
Intake Queue	Yes — <code>intakeGate.getIntakeQueue</code>	No	No	Count not displayed in header
Under Assessment	Yes — <code>getDashboardStats.workflowStateCounts</code>	No	No	Not surfaced
Internal Review	Yes — <code>getDashboardStats.workflowStateCounts</code>	No	No	Not surfaced
Technical Approval	Yes — <code>getDashboardStats.workflowStateCounts</code>	No	No	Not surfaced
Financial Decision	Yes — <code>byStatus({ status: "financial_decision" })</code>	No	No	Not surfaced
Repair Assigned	Yes — <code>getDashboardStats.workflowStateCounts</code>	No	No	Not surfaced

Assessment: The `getDashboardStats` procedure returns `workflowStateCounts` — a dictionary of claim counts by workflow state. This data is fetched on every dashboard load but is never rendered. The queue count column of the required matrix is available today with zero additional backend work. Average age per queue stage is available via `workflowAnalytics.getProcessingTimesByStage`, which queries the `workflow_audit_trail` table directly. SLA breach counts require a threshold configuration per stage, which does not currently exist but could be derived from the `getBottlenecks` procedure using the existing `threshold` parameter.

Gap Summary: Queue Health pillar is entirely absent from the UI. All required data is available in existing backend procedures.

2.2 Pillar 2 — Escalation Centre

Escalation Category	Backend Support	UI Implementation
High Value Pending Claims	<code>getManagerOverview</code> returns <code>totalSavings</code> and <code>totalAmt</code> ; <code>byStatus</code> can filter by amount	No dedicated escalation surface
High Fraud Risk Claims	<code>getFraudAlerts</code> returns all claims with <code>fraudRiskLevel</code> high/critical/elevated or score > 70	Fraud Alerts tab — present
Disputed Claims	<code>getEscalations</code> returns claims in <code>disputed</code> or <code>manual_review</code> workflow states	Not surfaced in Claims Manager Portal
Stuck Claims	<code>workflowAnalytics.getBottlenecks</code> identifies claims exceeding dwell thresholds	Not surfaced
Claims Exceeding SLA	<code>workflowAnalytics.getSLACompliance</code> exists	Not surfaced

Assessment: The Fraud Alerts tab provides partial escalation centre capability for fraud-risk claims. However, disputed claims, stuck claims, and SLA-breached claims have no visibility surface. The `getEscalations` procedure — which explicitly targets `disputed` and `manual_review` states — is not called from the Claims Manager Dashboard at all, despite being available and role-accessible.

2.3 Pillar 3 — Workflow Intelligence

Intelligence Metric	Backend Procedure	Dashboard Consumption
Workflow bottlenecks	<code>workflowAnalytics.getBottlenecks</code>	Not consumed
Queue accumulation	<code>getDashboardStats.workflowStateCounts</code>	Not rendered
Average dwell time per stage	<code>workflowAnalytics.getProcessingTimesByStage</code>	Not consumed
Maximum dwell time per stage	<code>workflowAnalytics.getBottlenecks</code> (returns <code>max_hours</code>)	Not consumed
Send-back frequency by stage	No dedicated procedure — derivable from <code>workflow_audit_trail</code>	Not available

Assessment: Four of five workflow intelligence metrics have existing backend support. Send-back frequency by stage is the only metric requiring a new query. The `workflow_audit_trail` table records every state transition with `previousState`, `newState`, `userId`, `userRole`, and `comments`. A send-back is definitionally any transition where `newState` is earlier in the workflow than `previousState`. This can be queried with a single SQL statement grouping by `previousState` (the stage from which the claim was sent back).

2.4 Pillar 4 — Workforce Intelligence

Metric	Procedure	Role Access	Dashboard Consumption
Processor: Claims assigned	<code>analytics.getUserProductivity</code>	<code>claims_manager</code>	Not consumed
Processor: Claims completed	<code>analytics.getUserProductivity</code>	<code>claims_manager</code>	Not consumed
Processor: SLA compliance	<code>analytics.getUserProductivity</code>	<code>claims_manager</code>	Not consumed
Processor: Reopen rate	Not available	N/A	Not available
Processor: Send-back rate	Not available	N/A	Not available
Assessor: Open assessments	<code>analytics.getAssessorPerformance</code>	<code>claims_manager</code>	Not consumed
Assessor: Completion time	<code>analytics.getAssessorPerformance</code>	<code>claims_manager</code>	Not consumed
Assessor: Accuracy score	<code>analytics.getAssessorPerformance</code>	<code>claims_manager</code>	Not consumed
Assessor: Performance tier	<code>analytics.getAssessorPerformance</code>	<code>claims_manager</code>	Not consumed

Assessment: The `analytics.getAssessorPerformance` and `analytics.getUserProductivity` procedures are both accessible to the `claims_manager` role and return the metrics required for workforce intelligence. Neither is called from the Claims Manager Dashboard. Processor reopen rate and send-back rate require new queries against the `workflow_audit_trail` table but are derivable from existing data.

2.5 Pillar 5 — Financial and Fraud Oversight

Oversight Metric	Backend Support	Dashboard Consumption	Notes
Leakage	<code>getManagerOverview</code> returns <code>savings</code> (est – approved delta)	Shown in KPI card as “Savings Identified”	Correctly scoped to operational oversight
Savings	<code>getManagerOverview.kpis.totalSavings</code>	Shown in KPI card	Present
Fraud Funnel	<code>analytics.getFraudInvestigationFunnel</code>	Not consumed	Procedure exists
Recovery Performance	<code>recovery.getKPIs</code>	Recovery KPI row — present	Counts only; no performance trend
High-Risk Approvals	<code>byStatus({ status: "financial_decision" })</code> + fraud score	Not surfaced as a combined view	Requires filter on Review Queue

Assessment: Savings and leakage metrics are correctly surfaced at the operational level. The Fraud Funnel — which shows the progression from flagged → investigated → confirmed → referred — is not surfaced despite the procedure existing. High-risk approvals (claims at `financial_decision` with high fraud scores) are not highlighted as a combined view, requiring the Claims Manager to cross-reference the Fraud Alerts tab with the Review Queue manually.

Phase 3 — Approval Governance Review

3.1 Technical Approval Workflow

The `approveClaim` procedure handles technical approval. It performs the following governance steps: fraud re-check at approval point (blocks `critical` fraud risk level), automation policy threshold check, `WorkflowEngine` state transition, `technicallyApprovedBy` and `technicallyApprovedAt` field update, and audit entry creation with action `claim_approved`.

Governance Assessment: The technical approval workflow is well-governed. The fraud re-check at approval point is a particularly strong control. The audit trail entry is clear and attributable.

Gap: The procedure transitions the claim to `repair_assigned` state, bypassing `financial_decision` for claims below the automation policy threshold. This is intentional but the audit trail comment reads “No financial approval required” without recording the threshold value that was applied. If the automation policy threshold changes, historical audit records will not reflect the threshold that was in force at the time of approval.

Recommendation: Record the `requireManagerApprovalAbove` threshold value in the audit trail `metadata` field at the time of approval.

3.2 Financial Decision Workflow

The `sendBackClaim` procedure (implemented in the v2.0 fix cycle) handles financial decision send-backs. It transitions `technical_approval` → `internal_review` or `financial_decision` → `technical_approval` via the `WorkflowEngine` with mandatory comment capture.

Governance Assessment: The send-back workflow is now correctly governed. The mandatory comment requirement ensures reason capture. The `WorkflowEngine` validates the transition against `WORKFLOW_TRANSITIONS` before executing.

Gap: The `closeForProcessing` action in the Review Queue tab calls `trpc.claims.approveClaim` — a procedure designed for technical approval, not closure. This creates an audit trail entry with action `claim_approved` for what is operationally a closure decision. The audit trail is therefore ambiguous: `claim_approved` may represent either a technical approval or a closure approval depending on the workflow state at the time of the call.

3.3 Closure Approval Workflow

The `payment_authorized` → `closed` transition is governed by the `WorkflowEngine` and requires the `claims_manager` or `executive` role. However, the UI action that triggers closure (`closeForProcessing`) calls `approveClaim`, which transitions the claim to `repair_assigned` — not `closed`. The intended closure transition (`payment_authorized` → `closed`) is not triggered by any UI action in the Claims Manager Dashboard.

Governance Assessment: The closure workflow has a critical semantic mismatch. The UI label says “Close for Processing” but the backend action performs a technical approval transition. This is the `closeForProcessing` defect documented in v1.0 and confirmed as unresolved in v2.0.

3.4 Send Back Workflow

The `sendBackClaim` procedure is correctly implemented. The `handleSendBack` function in the frontend opens a dialog that captures `comments` (required) and `targetRole` (optional). The mutation calls `trpc.claims.sendBackClaim` directly.

Governance Assessment: Send-back governance is sound following the v2.0 fix. The mandatory comment requirement is enforced at both the UI level (required field) and the backend level (Zod validation with `min(10)`).

Gap: The `targetRole` field is optional and has no validation against the actual workflow transition being performed. A Claims Manager could specify `targetRole: "assessor_internal"` while sending back from `financial_decision` , which is not a valid target for that transition. The WorkflowEngine will correctly reject the invalid transition, but the error message returned to the user will be a technical state-machine error rather than a clear governance message.

3.5 Approval Governance Summary

Workflow	Governance Quality	Critical Gap
Technical Approval	Strong	Threshold not recorded in audit metadata
Financial Decision	Strong	None
Closure Approval	Defective	<code>closeForProcessing</code> calls <code>approveClaim</code> — wrong procedure
Send Back	Strong	<code>targetRole</code> not validated against transition rules
Escalation	Absent	No escalation procedure exists; “Escalate” button calls <code>sendBackClaim</code>

Phase 4 — Rework and Quality Analytics Review

4.1 Send-Back Reason Data Availability

The `workflow_audit_trail` table records every state transition. The `comments` field captures the reason text supplied at the time of the transition. The `sendBackClaim` procedure prefixes all send-back comments with `"SENT BACK BY CLAIMS MANAGER: "` , making send-back transitions identifiable by comment prefix in addition to state direction.

The `previousState` and `newState` fields allow programmatic identification of backward transitions (where `newState` is earlier in the workflow sequence than `previousState`). This means send-back frequency by stage, send-back reason text, and the identity of the sending user are all available in the existing `workflow_audit_trail` table.

4.2 Rework Analytics Gap

No dedicated send-back analytics procedure exists. The `workflowAnalytics` router contains `getProcessingTimesByStage` , `getBottlenecks` , `getSLACompliance` , and `getTransitionTrends` — but no procedure that specifically analyses backward transitions or rework rates.

The `getTransitionTrends` procedure counts all transitions grouped by period but does not distinguish forward from backward transitions. It cannot answer “which processors generate the most rework” or “which approval stages generate the most send-backs.”

4.3 Recommended Analytics Procedures

The following three analytics procedures are recommended. All are derivable from existing `workflow_audit_trail` data with no schema changes required.

Procedure 1: `workflowAnalytics.getSendBackAnalytics`

Query: Select all `workflow_audit_trail` records where `newState` is earlier in the workflow sequence than `previousState` , grouped by `previousState` (the stage from which the claim was sent back). Return count, percentage of total transitions, and top three reason categories derived from `comments` text.

Procedure 2: `workflowAnalytics.getReworkByUser`

Query: Join `workflow_audit_trail` with `users` on `userId` . Filter for backward transitions. Group by `userId` and `userRole` . Return rework count per user, rework rate (rework transitions / total transitions for that user), and average rework reason.

Procedure 3: `workflowAnalytics.getQualityTrend`

Query: Group backward transitions by month. Calculate monthly rework rate (backward transitions / total transitions). Return trend data for chart rendering.

4.4 Structured Send-Back Reason Capture

The current `sendBackClaim` procedure captures free-text comments. This is sufficient for audit trail purposes but insufficient for analytics. Rework analytics require structured reason categories.

Recommendation: Add a `sendBackReason` enum field to the `sendBackClaim` input schema with values: `incomplete_documentation`, `incorrect_assessment`, `fraud_concern`, `policy_exclusion`, `cost_discrepancy`, `additional_information_required`, `other`. Store the structured reason in the `metadata` JSON field of the `workflow_audit_trail` record. Retain the free-text `comments` field for narrative context.

Phase 5 — Actionability Review

5.1 Widget Actionability Matrix

Widget	Information Only	Actionable	Recommended Actions to Add
KPI Cards (Row 1)	Yes	No	Drill-down to filtered claim list on click
Recovery KPI Row	Yes	No	Link to Recovery Portal filtered by status
Intake Queue Tab	No	Yes (assign to processor)	Add bulk assignment capability
Review Queue Tab	No	Yes (close, send back, view)	Add per-claim report dropdown
Active Claims Tab	Partial	Partial (view, route)	Add escalate, reassign actions
Fraud Alerts Tab	No	Yes (review, escalate)	Escalate should call dedicated escalation procedure, not <code>sendBackClaim</code>
Processed Claims Tab	Yes	No	Add reopen capability for disputed claims
Fleet Approvals Tab	No	Yes (approve fleet claims)	Present
Recently Closed Claims card	Yes	No	Remove or merge into Processed Claims tab

5.2 Critical Actionability Gaps

Escalation Action Misuse: The “Escalate” button in the Fraud Alerts tab calls `handleSendBack`, which opens the send-back dialog and calls `sendBackClaim`. Escalation is not a send-back. Escalating a fraud claim should transition it to `disputed` or `manual_review` state and notify the Risk Manager — not send it back to the previous workflow stage. No dedicated escalation procedure exists.

Reopen Capability: The `WORKFLOW_TRANSITIONS` map permits `closed` → `disputed`. This transition is not exposed in any UI action. Claims Managers cannot reopen closed claims from the dashboard.

Bulk Operations: The Intake Queue tab supports single-claim assignment only. No bulk assignment or bulk routing capability exists.

Phase 6 — Report Alignment Review

6.1 Claims Manager Authorised Reports

The following reports are authorised for the `claims_manager` role in `reportDefinitions.ts`.

Report Key	Report Name	Correct Access Location	Current UI Location
<code>claim.assessment</code>	Claim Assessment Report	Review Queue — per-claim action	Not surfaced in Claims Manager Portal
<code>claim.forensic</code>	Forensic Investigation Report	Fraud Alerts tab — per-claim action	Not surfaced
<code>claim.audit_trail</code>	Claim Audit Trail	Review Queue — per-claim action	Not surfaced
<code>claim.cost_comparison</code>	Cost Comparison Report	Review Queue — per-claim action	Not surfaced
<code>portfolio.claims_operations</code>	Claims Operations Monthly Report	Dashboard header — primary report	Not surfaced
<code>portfolio.fraud_intelligence</code>	Fraud Intelligence Report	Fraud Alerts tab — section header	Not surfaced
<code>portfolio.assessor_performance</code>	Assessor Performance Report	Workforce Intelligence section	Not surfaced
<code>risk_manager_portfolio</code>	Risk Manager Portfolio	Reports Centre only	Not surfaced
<code>executive.claims_trend</code>	Claims Trend Report	Dashboard header — date-ranged	Not surfaced
<code>executive.financial_exposure</code>	Financial Exposure Report	Dashboard header — financial oversight	Not surfaced
<code>recovery.case_summary</code>	Recovery Case Summary	Recovery section	Not surfaced

Finding: Eleven reports are authorised for the `claims_manager` role. None are surfaced in the Claims Manager Portal. The portal has no `KingaReportButton` components, no report dropdown menus, and no Reports Centre link in its sidebar navigation.

6.2 Report Access Priority

Reports required during claim decisions must be accessible within claim workflows. The following prioritisation applies.

Immediate Priority (within claim workflow): `claim.assessment`, `claim.audit_trail`, `claim.cost_comparison` — these three reports are required at the point of the close-for-processing decision and must be accessible directly from the Review Queue claim card.

Section Priority (within dashboard section): `portfolio.fraud_intelligence` in the Fraud Alerts tab; `portfolio.assessor_performance` in the Workforce Intelligence section; `recovery.case_summary` in the Recovery section.

Dashboard Header Priority: `portfolio.claims_operations`, `executive.claims_trend`, `executive.financial_exposure` — accessible from the dashboard header for period-based reporting.

Phase 7 — Claims Operations Monthly Report Review

7.1 Report Structure Validation

The `portfolio.claims_operations` report is authorised for the `claims_manager` role. The report generator is implemented in the `reportDefinitions.ts` dispatcher. The required content sections are validated below.

Required Section	Present in Report	Data Source Available	Notes
Executive Summary	To be verified	<code>getManagerOverview</code>	Standard report header
Throughput Analysis	To be verified	<code>getDashboardStats</code>	Total, completed, rejected counts available
Claims Ageing	To be verified	<code>workflowAnalytics.getProcessingTimesByStage</code>	Avg dwell per stage available
Workflow Bottlenecks	To be verified	<code>workflowAnalytics.getBottlenecks</code>	Max dwell, affected claims available
SLA Performance	To be verified	<code>workflowAnalytics.getSLACompliance</code>	SLA compliance by stage available
Financial Performance	To be verified	<code>getManagerOverview.kpis.totalSavings</code>	Savings and leakage available
Fraud Intelligence	To be verified	<code>analytics.getFraudInvestigationFunnel</code>	Funnel data available
Workforce Performance	To be verified	<code>analytics.getUserProductivity</code> , <code>getAssessorPerformance</code>	Both procedures available
Panel Beater Performance	To be verified	<code>analytics.getPanelBeaterAnalytics</code>	Available
Assessor Performance	To be verified	<code>analytics.getAssessorPerformance</code>	Available
Recovery Oversight	To be verified	<code>recovery.getKPIs</code>	KPI counts available
AI Accuracy	To be verified	<code>analytics.getAIAccuracy</code> or equivalent	To be confirmed
Operational Intervention Register	Missing	<code>workflow_audit_trail</code>	No procedure extracts escalations, send-backs, SLA breaches, bottleneck stages, and management interventions as a combined register

7.2 Operational Intervention Register

The Operational Intervention Register is the most operationally significant section of the Claims Operations Monthly Report. It must record:

- Escalated Claims (claims transitioned to `disputed` or `manual_review`)
- Sent Back Claims (backward workflow transitions with reason)
- SLA Breaches (claims exceeding stage dwell thresholds)
- Bottleneck Stages (stages with average dwell above threshold)
- Management Interventions (executive overrides, manual fraud escalations)

All five data categories are available in the `workflow_audit_trail` table. The `executiveOverride` field flags management interventions. The `overrideReason` field captures the reason. Backward transitions identify send-backs. Claims in `disputed` or `manual_review` identify escalations.

Recommendation: Implement a `getOperationalInterventionRegister` procedure in the `workflowAnalytics` router that queries the `workflow_audit_trail` table for the reporting period and returns a structured register of all five intervention categories. This procedure should be called by the `portfolio.claims_operations` report generator.

Phase 8 — Claims Manager Dashboard Target State Architecture

8.1 Design Principles

The target state dashboard must preserve all existing working functionality. No existing tabs are removed. The architecture adds operational intelligence above the existing tab structure without displacing the claim-level workflow tools that already work well.

The design distinction between the Claims Manager Portal and the Executive Dashboard is maintained throughout. The Claims Manager sees operational metrics — queue counts, stage dwell times, workforce productivity, send-back rates. The Executive sees strategic metrics — portfolio performance, financial exposure, market trends. These are different data sets serving different decisions.

8.2 Target State Layout

Row 1 — Operational KPIs (replaces current 4-card KPI row)

Card	Metric	Source Procedure	Change from Current
Total Claims	Count with period delta	<code>getManagerOverview.kpis.totalClaims</code>	No change
Completion Rate	Completed / Total × 100	<code>getManagerOverview.kpis.completedClaims</code>	New calculation
Backlog	Active claims count	<code>getManagerOverview.kpis.activeClaims</code>	Rename from “Active”
Savings Identified	Est – Approved delta	<code>getManagerOverview.kpis.totalSavings</code>	No change

Row 2 — Claims Operations Intelligence Bar (new section)

This row replaces the current Recovery KPI row and expands it into a full-width intelligence bar with seven panels. Each panel is a compact card with a count, a trend indicator, and a drill-down link.

Panel	Metric	Source Procedure	Action on Click
Queue Health	Counts per stage	<code>getDashboardStats.workflowStateCounts</code>	Expand to queue health matrix
Escalation Centre	Disputed + stuck claims	<code>getEscalations</code>	Navigate to escalation list
Claims Ageing	Avg dwell per stage	<code>workflowAnalytics.getProcessingTimesByStage</code>	Expand to ageing table
Workflow Bottlenecks	Stages exceeding threshold	<code>workflowAnalytics.getBottlenecks</code>	Expand to bottleneck detail
Fraud Funnel	Flagged → Confirmed count	<code>analytics.getFraudInvestigationFunnel</code>	Navigate to Fraud Alerts tab
Recovery Overview	Open + demand sent	<code>recovery.getKPIs</code>	Navigate to Recovery Portal
Leakage Metrics	Savings rate %	<code>getManagerOverview.kpis.totalSavings</code>	Expand to financial detail

Row 3 — Existing Tab Structure (preserved, no changes)

The six existing tabs are preserved in their current form: Intake Queue, Review Queue, Active Claims, Fraud Alerts, Processed Claims, Fleet Approvals. Report buttons are added to the Review Queue and Fraud Alerts tabs as per the Report Alignment recommendations in Phase 6.

Row 4 — Workforce Intelligence (new section, below tabs)

This section is rendered as a collapsible panel below the tab structure to avoid overwhelming the primary workflow view.

Panel	Metrics	Source Procedure
Processor Performance	Claims assigned, completed, SLA compliance	<code>analytics.getUserProductivity</code>
Assessor Performance	Open assessments, completion time, accuracy score, performance tier	<code>analytics.getAssessorPerformance</code>
Workload Distribution	Claims per processor, claims per assessor	Derived from above procedures

8.3 Navigation Additions

The Claims Manager sidebar currently lacks a Fleet Approvals entry point. Fleet Approvals exists as a tab within the dashboard but has no sidebar navigation item. Users who navigate directly to the dashboard land on the Intake Queue tab and may not discover Fleet Approvals.

Recommendation: Add “Fleet Approvals” as a sidebar navigation item linking to `/insurer/claims-manager?tab=fleet-approvals`.

Phase 9 — Implementation Prioritisation

9.1 Critical — Production Blockers

Recommendation	Business Value	Engineering Effort	Backend Available	Dependencies
Fix <code>closeForProcessing</code> to call correct procedure	Eliminates ambiguous audit trail entries; correct governance for closure decisions	0.5 days	<code>payment_authorized</code> → <code>closed</code> transition exists in WorkflowEngine	None
Add dedicated escalation procedure	Fraud escalation currently misroutes claims via send-back; disputed state transition not exposed	1 day	<code>disputed</code> state and <code>getEscalations</code> procedure exist	None

9.2 High — Operational Control Improvements

Recommendation	Business Value	Engineering Effort	Backend Available	Dependencies
Add Queue Health panel (Row 2)	Answers “which queues are overloaded?” — the second most important daily question	0.5 days	<code>getDashboardStats.workflowStateCounts</code> already fetched	None
Add Escalation Centre panel (Row 2)	Surfaces disputed and stuck claims without requiring tab navigation	0.5 days	<code>getEscalations</code> procedure exists	None
Add Claims Ageing panel (Row 2)	Answers “which workflow stages are slowing down?”	0.5 days	<code>workflowAnalytics.getProcessingTimesByStage</code> exists	None
Add per-claim report buttons to Review Queue	Assessment, audit trail, and cost comparison reports required at decision point	1 day	All three report generators exist	None
Add Fleet Approvals to sidebar navigation	Discoverability — tab exists but is not navigable from sidebar	0.25 days	Tab exists	None
Add Workflow Bottlenecks panel (Row 2)	Proactive bottleneck identification	0.5 days	<code>workflowAnalytics.getBottlenecks</code> exists	None

9.3 Medium — Management Intelligence Improvements

Recommendation	Business Value	Engineering Effort	Backend Available	Dependencies
Add Workforce Intelligence section (Row 4)	Answers “which assessors/processors are overloaded?”	1.5 days	<code>getAssessorPerformance</code> and <code>getUserProductivity</code> exist	None
Add Fraud Funnel panel (Row 2)	Operational fraud oversight without Executive Portal access	0.5 days	<code>getFraudInvestigationFunnel</code> exists	None
Implement <code>getSendBackAnalytics</code> procedure	Answers “why are claims being sent back?” and “which processors generate rework?”	1 day	<code>workflow_audit_trail</code> data available	None
Add structured <code>sendBackReason</code> enum to send-back dialog	Enables rework analytics; improves reason capture quality	0.5 days	Schema change required (metadata field)	<code>getSer</code>
Implement <code>getOperationalInterventionRegister</code> procedure	Required for Claims Operations Monthly Report completeness	1 day	<code>workflow_audit_trail</code> data available	None
Add Recovery section report button (<code>recovery.case_summary</code>)	Recovery oversight without navigating to Recovery Portal	0.25 days	Report generator exists	None

9.4 Low — UX and Convenience Improvements

Recommendation	Business Value	Engineering Effort	Backend Available	Dependencies
Add KPI card drill-down links	Reduces navigation steps from KPI to claim list	0.5 days	Filtered claim queries exist	None
Add Recovery KPI row link to Recovery Portal	Reduces navigation steps for recovery intervention	0.25 days	Recovery Portal exists	None
Add reopen capability to Processed Claims tab	Enables <code>closed</code> → <code>disputed</code> transition from UI	0.5 days	Transition exists in <code>WorkflowEngine</code>	None
Record automation policy threshold in audit metadata	Improves audit trail completeness for historical review	0.25 days	<code>metadata</code> field exists in <code>workflow_audit_trail</code>	None
Validate <code>targetRole</code> against transition rules in send-back dialog	Prevents confusing error messages on invalid send-back targets	0.5 days	Transition map available in <code>WORKFLOW_TRANSITIONS</code>	None
Merge Recently Closed Claims card into Processed Claims tab	Removes redundant UI section	0.25 days	N/A	None

9.5 Total Engineering Estimate

Priority	Item Count	Total Effort
Critical	2	1.5 days
High	6	3.25 days
Medium	6	4.25 days
Low	6	2.25 days
Total	20	11.25 days

The Critical and High items together represent 4.75 days of engineering work and would transform the portal from a claims list viewer into a functional Claims Operations Command Centre.

Deliverable Summary

Deliverable	Phase	Status
Role Alignment Assessment	Phase 1	Complete
Command Centre Assessment	Phase 2	Complete
Queue Health Review	Phase 2.1	Complete
Escalation Centre Review	Phase 2.2	Complete
Workflow Intelligence Review	Phase 2.3	Complete
Workforce Intelligence Review	Phase 2.4	Complete
Approval Governance Review	Phase 3	Complete
Rework and Quality Analytics Review	Phase 4	Complete
Actionability Review	Phase 5	Complete
Report Alignment Matrix	Phase 6	Complete
Claims Operations Monthly Report Review	Phase 7	Complete
Target State Dashboard Architecture	Phase 8	Complete
Prioritised Implementation Plan	Phase 9	Complete
Revised Production Readiness Verdict	Below	Complete

Revised Production Readiness Verdict

Score: 5.8 / 10 — Operationally Functional, Not Operationally Intelligent

The Claims Manager Portal is production-ready for individual claim processing and approval governance. The workflow engine is correctly integrated, the send-back mechanism is now properly governed, and the fraud alert visibility is appropriate for the role. A Claims Manager can process claims, approve decisions, and review fraud alerts using the current portal.

The portal is not yet production-ready as a Claims Operations Command Centre. A Claims Manager cannot answer six of the ten core operational questions from the current dashboard. Queue health, SLA compliance, workflow bottlenecks, and workforce performance are invisible. The analytics infrastructure to support all of these exists in the backend and is simply not surfaced.

The path from the current state to a command centre is well-defined and achievable within the 11.25-day engineering estimate above. The Critical and High items — representing 4.75 days — are sufficient to achieve command centre status.

Recommended Go-Live Condition: Complete all Critical and High items before production deployment. The portal should not be presented to Claims Managers as their primary operational tool until queue health, escalation centre, and claims ageing visibility are implemented. These are not enhancements — they are the core operational intelligence that defines the role.

End of Document — KINGA Claims Manager Portal Alignment and Redesign Audit v3.0